

Language choice workshop

Revise 360 lesson plan — 60 minutes

Unit	OCR J277 2.5 Programming languages and IDEs — Paper 2, Computational thinking, algorithms and programming
Lesson	Language choice workshop (lesson 2 of the unit)
Lesson ID	p1-l02
Experience ID	p1-l02
Plan ID	PLAN-PL-L02
Content version	ff726435bb0c
Website	https://revise360.co.uk/experience.html?id=p1-l02

Big question, outcomes and success criteria

Big question — Why might a school app and a small hardware routine need different language choices?

Prerequisites

The lesson before this one is p1-l01. Pupils should already have finished it: the starter below retrieves from it.

Assessable outcomes

1. **p1-l02-o1** — Compare development effort at different language levels.
2. **p1-l02-o2** — Explain portability and hardware control.
3. **p1-l02-o3** — Justify language choices using project requirements.

Vocabulary, resources and preparation

Key vocabulary — Portability, Maintainability

Resources

SLIDES	presentations/PL_L02_InstructionWorkshop_Lesson.pptx
WORKSHEET	worksheets/PL_L02_InstructionWorkshop_Worksheet.docx
ONLINE EXPERIENCE	experience.html?id=p1-l02

Before the lesson

1. Sign in on the **teacher dashboard** (teacher.html) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the 360 image is already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.

3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build APP_CONFIG.backendUrl is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on PL_L02_InstructionWorkshop_Worksheet.docx is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong.	Correct their own starter.	A correction written, not just a tick.
8-18	Explain and model	The new idea in two or three steps, with the example worked in front of the class.	Listen and answer questions about the example.	They can say why, not only what.
18-21	Set up and settle	Sign-in, launch, and what to write before anyone taps an answer.	Sign in, open the lesson, find Reading and changing code.	Everyone is in and on station 1.
21-43	Online: work the stations in order	Pupils read each wall, write the key fact and the challenge, then answer at the badge.	Read each wall, write the key fact and the challenge, then answer at the badge.	The worksheet has writing on it for each of the 6 stations before the badge is tapped.
43-48	Check understanding	The hinge question, hands down.	Answer the hinge question, hands down.	The split across the options.
48-56	Apply	Exam practice on the worksheet, in silence.	Exam practice, in silence.	worksheet, Exam practice a) to c)
56-60	Exit and set up the next lesson	The exit question and the retrieval link to next lesson.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. Reading and changing code

- High-level code uses readable words and structures.
- Readable code is usually easier to understand and change.
- Low-level code requires more knowledge of the processor.

Challenge on this wall: Why might a high-level language help staff update a quiz each week?

2. Development time

- High-level operations can express a task in fewer instructions.

- Low-level code often needs more detailed instructions.
- More detailed code can take longer to write and check.

Challenge on this wall: Why can high-level code reduce development time?

3. Moving to another system

- High-level source is usually more portable.
- The new system needs a suitable translator.
- Low-level code is tied more closely to a processor.

Challenge on this wall: Why may low-level code need rewriting for a different processor?

4. Control of hardware

- Low-level code closely matches processor operations.
- It allows detailed control of hardware and memory use.
- This control comes with greater development effort.

Challenge on this wall: Why might low-level code suit a routine needing precise hardware control?

5. Speed and memory

- Low-level code can be carefully tuned for a processor.
- This can help control execution speed and memory use.
- High-level code usually makes development easier.

Challenge on this wall: A tiny routine must use little memory. Give one benefit and one drawback of low-level code.

6. A justified recommendation

- Identify the main requirement in the project.
- Choose the language level that addresses it.
- Explain both the benefit and a relevant drawback.

Challenge on this wall: Recommend a level for a school app that needs frequent changes on several systems.

Retrieval and the check for understanding

Starter — Which is usually easier for a person to understand: high-level source or binary machine code? Give a reason.

On the board or the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (p1 - 102 - s4 - c1), so the wording a pupil sees matches the wording you say.

Which requirement most strongly favours low-level code?

Correct: Precise control over processor operations

What a pupil might choose instead

- Readable code for weekly staff edits
- Minimum processor knowledge
- Rapid writing of a general school app

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: Low-level languages expose hardware operations in more detail.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
Recognise	12	The answer is on the screen and the pupil picks it: multiple choice, choosing several, matching a pair.
Construct	2	Nothing is offered. The pupil classifies, orders, fills in a table or works a number out, and types or places the answer.

Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	Reading and changing code	2	Why might a high-level language help staff update a quiz each week?
2	Development time	2	Why can high-level code reduce development time?
3	Moving to another system	2	Why may low-level code need rewriting for a different processor?
4	Control of hardware	2	Why might low-level code suit a routine needing precise hardware control?
5	Speed and memory	2	A tiny routine must use little memory. Give one benefit and one drawback of low-level code.

#	STATION	ACTIVITIES	RECORD ON PAPER
6	A justified recommendation	2	Recommend a level for a school app that needs frequent changes on several systems.
final	Final challenge	2	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
p1-102-x1	Explain two benefits of a high-level language for an app that needs regular updates on different systems.	4	worksheet, Exam practice a); deck, Exam practice slide
p1-102-x2	A small routine needs precise hardware control. Explain one benefit and one drawback of low-level code.	4	worksheet, Exam practice b); deck, Exam practice slide
p1-102-x3	Explain why portable high-level source still needs translation.	2	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/p1-102.json`, which is not published with the website.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **Which priority most strongly supports high-level code for a changing s** — Staff need binary instructions on paper; The code must never be translated
- **Why can high-level code be quicker to develop?** — It guarantees there are no errors; It never needs a translator
- **What must be checked when moving high-level source to another system?** — The keyboard has the same colour; Every CPU uses identical instructions
- **Which requirement most strongly favours low-level code?** — Readable code for weekly staff edits; Minimum processor knowledge
- **Which benefit fits a routine with strict memory limits?** — Guaranteed zero memory use; No need to understand hardware

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- Compare development effort at different language levels.
- Explain portability and hardware control.
- Justify language choices using project requirements.

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is p1 - 103, Translation studio. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.